perform actions on the object, and serialize the object, sending it in response. Consider the main objects in RiffMates: to build a full API, you need endpoints for musicians, bands, venues, and promoters. That's a lot of boilerplate code.

Enter Django Ninja (sneakily, of course—it is a ninja). Ninja (https://django-ninja.dev/) is a third-party library that reduces the amount of boilerplate code you need for your API endpoints. You turn a view into an endpoint using a decorator, and the serialization of objects into JSON is mostly done for you. Ninja knows how to serialize your Model classes and can use Python type hints to interact with non-Model fields as well. If you've ever used the FastAPI (https://fastapi.tiangolo.com/) web framework, you'll find Ninja familiar; it had heavy influence on the library's design.

To get started with Ninja, you'll need to install it. For full details on installing third party apps, see appendix A. Ninja is one of the simpler libraries to install; all you need to do is run `pip install django-ninja` and you're good to go. It is a rare Django library that technically isn't an app and so does not need to be registered in `INSTALLED_APPS`.

To build an API, you instantiate a `NinjaAPI` object. This should only be done once (or once per API, more on that later). Groups of API endpoints get collected in a `Router` object, and each router gets registered with the `NinjaAPI` object. It is called a *router*, as it is collecting view routes, just like your other views registered in urls.py.

The main Ninja object has a `.urls` property, which you use inside a `path` object in your urls.py file. `Router` object instances have methods that correspond to the HTTP methods and get used as decorators. The decorators turn view functions into API endpoints.

Let's start building an API by adding a `"Hello, world"`-inspired endpoint. Inside your RiffMates/home directory, create a new file called api.py. Inside this file, create a Ninja `Router` object, a view function that returns a string, and wrap the function with the `.get()` method of the router. The `.get()` method takes an argument: the last part of the URL for this API endpoint. The following listing contains this code.

Listing 11.1 An API endpoint that returns a string

```
# RiffMates/home/api.py
from ninja import Router

router = Router()

@router.get("/")
def home(request):
    return "RiffMates rocks!"
```

Each api.py file gets its own Router instance that groups all the API endpoints in the file.

A Router instance contains methods for each HTTP method. You decorate your view with one such method, passing in the end part of the URL for this endpoint.

Ninja automatically serializes returned values as a JSON result.

Now that you have declared an endpoint, you need to register it. Open RiffMates/urls.py, and then add the code in the following listing.